



POLITÉCNICA

UNIVERSIDAD
POLITÉCNICA
DE MADRID



Universidad
Politécnica
de Madrid

ETSI SISTEMAS
INFORMÁTICOS

Práctica 1: Lectura y adquisición de datos de un receptor GPS

Grupo 10

Integrantes: ALONSO BAYARRIZ ALVARO, CARRION HUARACA KEVIN ANDRES, CONDE MARIA JAVIER, MARTINEZ MANZANO YZAN, RAMOS CASTRO PABLO

Universidad Politécnica de Madrid

E.T.S. de Ingeniería de Sistemas Informáticos

1. Introducción y Objetivos

Los sistemas de posicionamiento global (GPS) son el pilar fundamental de la navegación moderna. El objetivo principal de esta práctica es diseñar y desarrollar una aplicación en Python capaz de establecer comunicación con un receptor GPS a través del puerto serie, adquiriendo y procesando en tiempo real las tramas estándar NMEA y convertirlos en coordenadas UTM.

El proyecto se centra específicamente en la captura de la trama GGA, la cual contiene la información geoespacial esencial calculada por el receptor. A partir de estos datos, el sistema no solo extrae las coordenadas geográficas, pues implementa matemáticamente la transformación de dichas coordenadas geográficas a coordenadas cartesianas UTM.

Requisitos principales del enunciado:

- Lectura de datos del receptor GPS por puerto serie.
- Adquisición de la trama GGA (posición en latitud/longitud).
- Transformación a coordenadas UTM (X, Y) mediante Gauss-Krüger / Transversa de Mercator.
- Receptor configurado a 1 Hz en el laboratorio.

2. Arquitectura concurrente (Multithreading) y Descripción del Sistema

La lectura del puerto serie es una operación bloqueante. Para mantener una ejecución fluida, el programa se divide en dos hilos: un hilo lector (`gps_reader`) que lee y filtra GGA y un hilo principal que consume una cola (`queue.Queue`) para realizar conversiones y mostrar la salida. Esta separación desacopla la velocidad de adquisición de la de procesado y evita congelaciones en la consola.

El flujo de trabajo del programa se ejecuta en un proceso continuo que abarca: la configuración y lectura del puerto serie en segundo plano; el filtrado y extracción de datos exclusivos de las tramas GGA; la conversión matemática de la codificación NMEA a grados decimales; y la transformación espacial a coordenadas cartesianas UTM para la monitorización de los resultados en tiempo real.

Diagrama de componentes:

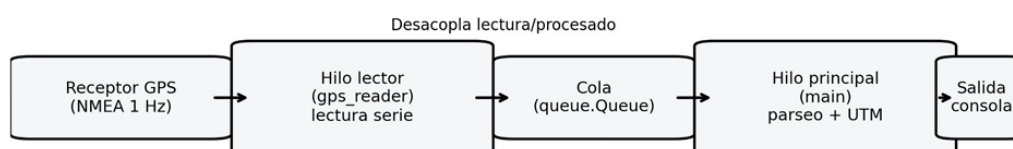


Figura 1. Flujo de datos y desacoplo lectura/procesado mediante cola.

3. Desarrollo del Proyecto

3.1. Configuración de la Comunicación Serie

La interfaz de comunicación con el receptor GPS se ha implementado mediante la librería pyserial. Se ha configurado una conexión síncrona en el puerto COM3 con los siguientes parámetros técnicos: 4800 baudios, 8 bits de datos, sin paridad, 1 bit de parada y sin control de flujo. El código incluye manejo de excepciones (try-except) para gestionar de forma segura posibles errores de conexión y garantizar un cierre limpio si el hardware no está disponible.

Parámetro	Valor
Puerto	COM3 (editable en el código)
Baudios	4800 bps
Bits de datos	8
Paridad	Ninguna
Bits de parada	1
Control de flujo	Ninguno
Timeout lectura	1 s

3.2. Adquisición de Datos y Multithreading

Para evitar que la lectura bloqueante del puerto serie congele el programa, se ha implementado una arquitectura concurrente dividida en dos flujos: Hilo Lector (gps_reader): Escucha en segundo plano el puerto serie. Al detectar y validar una trama GGA, la empaqueta y la introduce en una cola de datos.

Hilo Principal: Extrae los datos de la cola, realiza los cálculos de conversión y muestra la información en pantalla. Esta separación hace el código más eficiente, modular e incorpora pausas controladas (sleep) para optimizar el consumo de CPU.

3.3. Procesamiento de la Trama GGA y Conversión a Grados Decimales

El hilo lector filtra las líneas buscando los prefijos \$GPGGA o \$GNGGA. Al validar una trama, el sistema utiliza la coma (,) como delimitador para segmentar el string y extraer: latitud, hemisferio (N/S), longitud, hemisferio (E/W), calidad del posicionamiento, número de satélites en uso y altitud sobre el nivel del mar. Si la trama carece de coordenadas (pérdida de cobertura), es descartada automáticamente.

Dado que el formato NMEA expresa las coordenadas en grados y minutos, el programa las convierte a grados decimales sumando los grados a los minutos divididos por 60. Finalmente, asigna valores negativos a las latitudes Sur (S) y longitudes Oeste (W) para mantener el rigor posicional.

3.4. Transformación de Coordenadas Geográficas a UTM

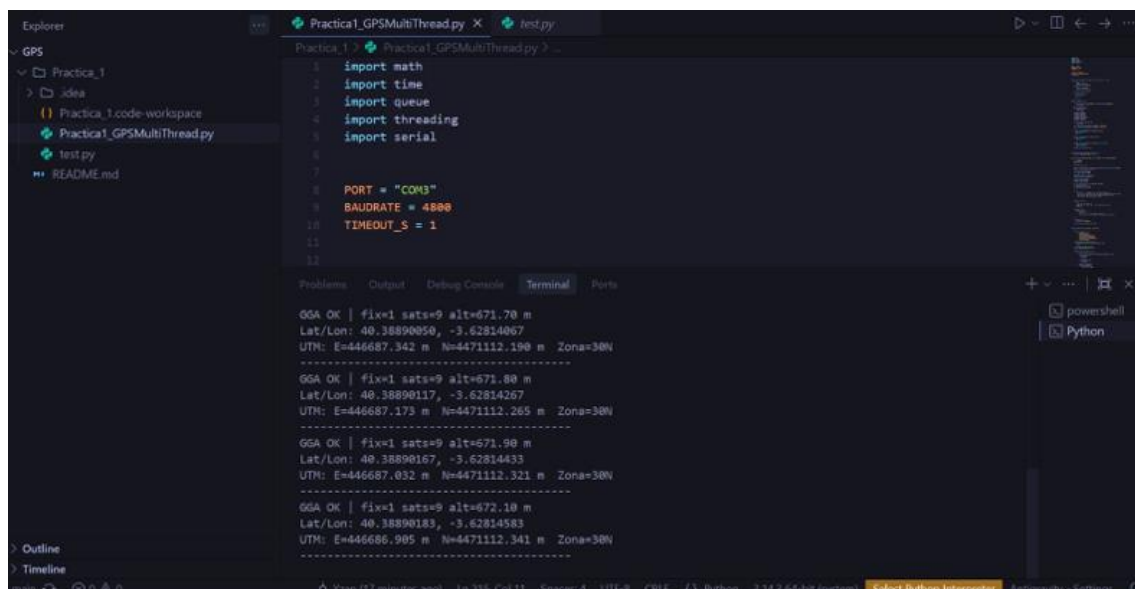
Una vez obtenidas latitud y longitud en grados decimales, se proyecta la posición a UTM. En lugar de depender de librerías externas, el programa implementa analíticamente las ecuaciones de la proyección Transversa de Mercator. Para ello, se utilizan las constantes oficiales del elipsoide de referencia WGS84:

Parámetro	Valor
Semieje mayor a (WGS84)	6378137.0 m
Excentricidad al cuadrado e^2	0.00669437999013
Factor de escala k0	0.9996
Falso Este	500000 m
Falso Norte (hemisferio S)	10000000 m

El algoritmo calcula de forma dinámica el huso horario (zona UTM) correspondiente en función de la longitud actual. A continuación, mediante el cálculo de la longitud del arco de meridiano y los radios de curvatura, se obtienen las coordenadas proyectadas: Este (E) y Norte (N) expresadas en metros.

4. Pruebas y resultados

La prueba de ejecución se realizó con el receptor GPS del laboratorio conectado por puerto serie. El programa muestra en consola la calidad del fix (fix), el número de satélites utilizados (sats), la altitud y la posición tanto en latitud/longitud (grados decimales) como en UTM (E, N).



```
1 import math
2 import time
3 import queue
4 import threading
5 import serial
6
7
8 PORT = "COM3"
9 BAUDRATE = 4800
10 TIMEOUT_S = 1
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

Terminal Output:

```
GGA OK | fix=1 sats=9 alt=671.70 m
Lat/Lon: 40.38890050, -3.62814067
UTM: E=446687.342 m N=4471112.190 m Zona=30N
-----
GGA OK | fix=1 sats=9 alt=671.80 m
Lat/Lon: 40.38890117, -3.62814267
UTM: E=446687.173 m N=4471112.265 m Zona=30N
-----
GGA OK | fix=1 sats=9 alt=671.90 m
Lat/Lon: 40.38890167, -3.62814433
UTM: E=446687.032 m N=4471112.321 m Zona=30N
-----
GGA OK | fix=1 sats=9 alt=672.10 m
Lat/Lon: 40.38890183, -3.62814583
UTM: E=446686.905 m N=4471112.341 m Zona=30N
-----
```

Figura 2. Salida en consola durante la adquisición de tramas GGA y conversión a UTM

Comprobaciones básicas realizadas:

- Orden de magnitud UTM coherente en España: $E \approx 400\text{--}600\text{ km}$ y $N \approx 4\text{--}5$ millones de metros.
- La zona UTM calculada coincide con la esperada en el entorno de prueba (30N en Madrid).
- Gestión de ausencia de datos: si no llegan líneas (timeout), el hilo lector continúa sin bloquear el programa.

5. Instrucciones de ejecución y resolución de problemas

Ejecución:

1. Instalar dependencias: `pip install pyserial`
2. Conectar el receptor GPS y localizar el puerto COMx en el Administrador de dispositivos.
3. Modificar PORT en el script si el puerto no es COM3.
4. Ejecutar: `python Practica1_GPS.py`

6. Conclusiones

Se ha logrado adquirir telemetría de un receptor GPS físico mediante puerto serie, aislar la información relevante de la sentencia GGA y transformar las coordenadas geodésicas a coordenadas planas UTM bajo el sistema WGS84. Asimismo, la integración del multithreading garantiza un flujo de ejecución ininterrumpido y eficiente, estableciendo una base de código altamente profesional y fácilmente escalable